

3.2 RANDOM MATRIX INJECTION

In computation graphs consisting of vector operations, the vectorized computation graph is a more compact representation. We introduce an alternative view on sampling paths in this case. A single path in the vectorized computation graph represents many paths in the underlying scalar computation graph. As an example, Figure 2c is a vector representation for Figure 2b. For this example,

$$\frac{\partial y}{\partial \theta} = \frac{\partial y}{\partial C} \frac{\partial C}{\partial B} \frac{\partial B}{\partial A} \frac{\partial A}{\partial \theta} \quad (2)$$

where A, B, C are vectors with entries a_i, b_i, c_i , $\partial C/\partial B, \partial B/\partial A$ are 3×3 Jacobian matrices for the intermediate operations, $\partial y/\partial C$ is 1×3 , and $\partial A/\partial \theta$ is 3×1 .

We now note that the contribution of the path $p = \theta \rightarrow a_1 \rightarrow b_2 \rightarrow c_2 \rightarrow y$ to the gradient is,

$$\frac{\partial y}{\partial C} P_2 \frac{\partial C}{\partial B} P_2 \frac{\partial B}{\partial A} P_1 \frac{\partial A}{\partial \theta} \quad (3)$$

where $P_i = e_i e_i^T$ (outer product of standard basis vectors). Sampling from $\{P_1, P_2, P_3\}$ and right multiplying a Jacobian is equivalent to sampling the paths passing through a vertex in the scalar graph.

In general, if we have transition $B \rightarrow C$ in a vectorized computational graph, where $B \in \mathbb{R}^d, C \in \mathbb{R}^m$, we can insert a random matrix $P = d/k \sum_{s=1}^k P_s$ where each P_s is sampled uniformly from $\{P_1, P_2, \dots, P_d\}$. With this construction, $\mathbb{E}P = I_d$, so

$$\mathbb{E} \left[\frac{\partial C}{\partial B} P \right] = \frac{\partial C}{\partial B}. \quad (4)$$

If we have a matrix chain product, we can use the fact that the expectation of a product of independent random variables is equal to the product of their expectations, so drawing independent random matrices P_B, P_C would give

$$\mathbb{E} \left[\frac{\partial y}{\partial C} P_C \frac{\partial C}{\partial B} P_B \right] = \frac{\partial y}{\partial C} \mathbb{E}[P_C] \frac{\partial C}{\partial B} \mathbb{E}[P_B] = \frac{\partial y}{\partial C} \frac{\partial C}{\partial B} \quad (5)$$

Right multiplication by P may be achieved by sampling the intermediate Jacobian: one does not need to actually assemble and multiply the two matrices. For clarity we adopt the notation $\mathcal{S}_P[\partial C/\partial B] = \partial C/\partial B P$. This is sampling (with replacement) k out of the d vertices represented by B , and only considering paths that pass from those vertices.

The important properties of P that enable memory savings with an unbiased approximation are

$$\mathbb{E}P = I_d \quad \text{and} \quad P = RR^T, R \in \mathbb{R}^{d \times k}, k < d. \quad (6)$$

We could therefore consider other matrices with the same properties. In our additional experiments in the appendix, we also let R be a random projection matrix of independent Rademacher random variables, a construction common in compressed sensing and randomized dimensionality reduction.

In vectorized computational graphs, we can imagine a two-level sampling scheme. We can both sample paths from the computational graph where each vertex on the path corresponds to a vector. We can also sample within each vector path, with sampling performed via matrix injection as above.

In many situations the full intermediate Jacobian for a vector operation is unreasonable to store. Consider the operation $B \rightarrow C$ where $B, C \in \mathbb{R}^d$. The Jacobian is $d \times d$. Thankfully many common operations are element-wise, leading to a diagonal Jacobian that can be stored as a d -vector. Another common operation is matrix-vector products. Consider $Ab = c$, $\partial c/\partial b = A$. Although A has many more entries than c or b , in many applications A is either a parameter to be optimized or is easily recomputed. Therefore in our implementations, we do not directly construct and sparsify the Jacobians. We instead sparsify the input vectors or the compact version of the Jacobian in a way that has the same effect. Unfortunately, there are some practical operations such as softmax that do not have a compactly-representable Jacobian and for which this is not possible.

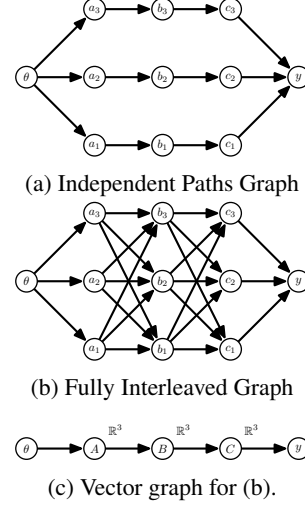


Figure 2: Common computational graph patterns. The graphs may be arbitrarily deep and wide. (a) A small number of independent paths. Path sampling has constant variance with depth. (b) The number of paths increases exponentially with depth; path sampling gives high variance. Independent paths are common when a loss decomposes over data. Fully interleaved graphs are common with vector operations.

3.3 VARIANCE

The variance incurred by path sampling and random matrix injection will depend on the structure of the LCG. We present two extremes in Figure 2. In Figure 2a, each path is independent and there are a small number of paths. If we sample a fixed fraction of all paths, variance will be constant in the depth of the graph. In contrast, in Figure 2b, the paths overlap, and the number of paths increases exponentially with depth. Sampling a fixed fraction of all paths would require almost all edges in the graph, and sampling a fixed fraction of vertices at each layer (using random matrix injection, as an example) would lead to exponentially increasing variance with depth.

It is thus difficult to apply sampling schemes without knowledge of the underlying graph. Indeed, our initial efforts to apply random matrix injection schemes to neural network graphs resulted in variance exponential with depth of the network, which prevented stochastic optimization from converging. We develop tailored sampling strategies for computation graphs corresponding to problems of common interest, exploiting properties of these graphs to avoid the exploding variance problem.

4 CASE STUDY: NEURAL NETWORKS

We consider neural networks composed of fully connected layers, convolution layers, ReLU nonlinearities, and pooling layers. We take advantage of the important property that many of the intermediate Jacobians can be compactly stored, and the memory required during reverse-mode is often bottlenecked by a few operations. We draw a vectorized computational graph for a typical simple neural network in figure 3. Although the diagram depicts a dataset of size of 3, mini-batch size of size 1, and 2 hidden layers, we assume the dataset size is N . Our analysis is valid for any number of hidden layers, and also recurrent networks. We are interested in the gradients $\partial y / \partial W_1$ and $\partial y / \partial W_2$.

4.1 MINIBATCH SGD AS RANDOMIZED AD

At first look, the diagram has a very similar pattern to that of 2a, so that path sampling would be a good fit. Indeed, we could sample $B < N$ paths from W_1 to y , and also B paths from W_2 to y . Each path corresponds to processing a different mini-batch element, and the computations are independent.

In empirical risk minimization, the final loss function is an average of the loss over data points. Therefore, the intermediate partials $\partial y / \partial h_{2,x}$ for each data point x will be independent of the other data points. As a result, if the same paths are chosen in path sampling for W_1 and W_2 , and if we are only interested in the stochastic gradient (and not the full function evaluation), the computation graph only needs to be evaluated for the data points corresponding to the sampled paths. This exactly corresponds to mini-batching. The paths are visually depicted in Figure 3b.

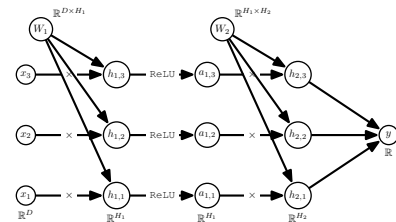
4.2 ALTERNATIVE SGD SCHEMES WITH RANDOMIZED AD

We wish to use our principles to derive a randomization scheme that can be used on top of mini-batch SGD. We ensure our estimator is unbiased as we randomize by applying random matrix injection independently to various intermediate Jacobians. Consider a path corresponding to data point 1. The contribution to the gradient $\partial y / \partial W_1$ is

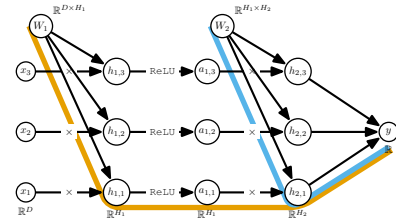
$$\frac{\partial y}{\partial h_{2,1}} \frac{\partial h_{2,1}}{\partial a_{1,1}} \frac{\partial a_{1,1}}{\partial h_{1,1}} \frac{\partial h_{1,1}}{\partial W_1} \quad (7)$$

Using random matrix injection to sample every Jacobian would lead to exploding variance. Instead, we analyze each term to see which are memory bottlenecks.

$\partial y / \partial h_{2,1}$ is the Jacobian with respect to (typically) the loss. Memory requirements for this Jacobian are independent of depth of the network. The dimension of the classifier is usually smaller (10 – 1000) than the other layers (which can have dimension 10,000 or more in convolutional networks). Therefore, the Jacobian at the output layer is not a memory bottleneck.



(a) Neural network computational graph



(b) Computational Graph with Mini-batching

Figure 3: NN computation graphs.

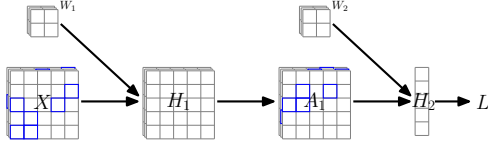


Figure 4: Convnet activation sampling for one mini-batch element. X is the image, H is the pre-activation, and A is the activation. A is the output of a ReLU, so we can store the Jacobian $\partial A_1 / \partial H_1$ with 1 bit per entry. For X and H we sample spatial elements and compute the Jacobians $\partial H_1 / \partial W_1$ and $\partial H_2 / \partial W_2$ with the sparse tensors.

$\partial h_{2,1} / \partial a_{1,1}$ is the Jacobian of the hidden layer with respect to the previous layer activation. This can be constructed from W_2 , which must be stored in memory, with memory cost independent of mini-batch size. In convnets, due to weight sharing, the effective dimensionality is much smaller than $H_1 \times H_2$. In recurrent networks, it is shared across timesteps. Therefore, these are not a memory bottleneck.

$\partial a_{1,1} / \partial h_{1,1}$ contains the Jacobian of the ReLU activation function. This can be compactly stored using 1-bit per entry, as the gradient can only be 1 or 0. Note that this is true for ReLU activations in particular, and not true for general activation functions, although ReLU is widely used in deep learning. For ReLU activations, these partials are not a memory bottleneck.

$\partial h_{1,1} / \partial W_1$ contains the memory bottleneck for typical ReLU neural networks. This is the Jacobian of the hidden layer output with respect to W_1 , which, in a multi-layer perceptron, is equal to x_1 . For B data points, this is a $B \times D$ dimensional matrix.

Accordingly, we choose to sample $\partial h_{1,1} / \partial W_1$, replacing the matrix chain with $\frac{\partial y}{\partial h_{2,1}} \frac{\partial h_{2,1}}{\partial a_{1,1}} \frac{\partial a_{1,1}}{\partial h_{1,1}} \mathcal{S}_{P_{W_1}} \left[\frac{\partial h_{1,1}}{\partial W_1} \right]$. For an arbitrarily deep NN, this can be generalized:

$$\frac{\partial y}{\partial h_{d,1}} \frac{\partial h_{d,1}}{\partial a_{d-1,1}} \frac{\partial a_{d-1,1}}{\partial h_{d-1,1}} \dots \frac{\partial a_{1,1}}{\partial h_{1,1}} \mathcal{S}_{P_{W_1}} \left[\frac{\partial h_{1,1}}{\partial W_1} \right], \quad \frac{\partial y}{\partial h_{d,1}} \frac{\partial h_{d,1}}{\partial a_{d-1,1}} \frac{\partial a_{d-1,1}}{\partial h_{d-1,1}} \dots \frac{\partial a_{2,1}}{\partial h_{2,1}} \mathcal{S}_{P_{W_2}} \left[\frac{\partial h_{2,1}}{\partial W_2} \right]$$

This can be interpreted as sampling activations on the backward pass. This is our proposed alternative SGD scheme for neural networks: along with sampling data points, we can also sample activations, while maintaining an unbiased approximation to the gradient. This does not lead to exploding variance, as along any path from a given neural network parameter to the loss, the sampling operation is only applied to a single Jacobian. Sampling for convolutional networks is visualized in Figure 4.

4.3 NEURAL NETWORK EXPERIMENTS

We evaluate our proposed RAD method on two feedforward architectures: a small fully connected network trained on MNIST, and a small convolutional network trained on CIFAR-10. We also evaluate our method on an RNN trained on Sequential-MNIST. The exact architectures and the calculations for the associated memory savings from our method are available in the appendix. In Figure 5 we include empirical analysis of gradient noise caused by RAD vs mini-batching.

We are mainly interested in the following question:

For a fixed memory budget and fixed number of gradient descent iterations, how quickly does our proposed method optimize the training loss compared to standard SGD with a smaller mini-batch?

Reducing the mini-batch size will also reduce computational costs, while RAD will only reduce memory costs. Theoretically our method could reduce computational costs slightly, but this is not our focus. We only consider the memory/gradient variance tradeoff while avoiding adding significant overhead on top of vanilla reverse-mode (as is the case for checkpointing).

Results are shown in Figure 6. Our feedforward network full-memory baseline is trained with a mini-batch size of 150. For RAD we keep a mini-batch size of 150, and try 2 different configurations. For "same sample", we sample with replacement a 0.1 fraction of activations, and the same activations are sampled for each mini-batch element. For "different sample", we sample a 0.1 fraction of activations, independently for each mini-batch element. Our "reduced batch" experiment is trained without RAD with a mini-batch size of 20 for CIFAR-10 and 22 for MNIST. This achieves similar memory budget as RAD with mini-batch size 150. Details of this calculation and of hyperparameters are in the appendix.